

Figures

```
rm(list=ls())
```

```
f <- function(x,r,u,b,c){
```

```
  ff=r*((2*b^2*c*x^(-3))*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1))/(1-(1-(1-exp(-b^2*x^(-2)))^c))^2)*exp(u*((1-(1-exp(-b^2*x^(-2)))^c)/(1-(1-(1-exp(-b^2*x^(-2)))^c))))*exp(-(r/u)*(exp(u*(1-(1-exp(-b^2*x^(-2)))^c)/(1-(1-(1-exp(-b^2*x^(-2)))^c)))-1))

}
```

```
F <- function(x,r,u,b,c){
```

```
  ff = 1-exp(-(r/u)*(exp(u*(1-(1-exp(-b^2*x^(-2)))^c)/(1-(1-(1-exp(-b^2*x^(-2)))^c)))-1))
```

```
h<-function(x,r,u,b,c){
```

```
  r*u*(c*b*(x^(-c-1))*exp(-b*x^(-c))*(((exp(-b*x^(-c)))/(1-(exp(-b*x^(-c)))))-log(1-(exp(-b*x^(-c)))))*(-(exp(-b*x^(-c)))*log(1-(exp(-b*x^(-c)))))^(u-1)

}
```

```
S<-function(x,r,u,b,c){
```

```
  exp(-(r/u)*(exp(u*(1-(1-exp(-b^2*x^(-2)))^c)/(1-(1-(1-exp(-b^2*x^(-2)))^c)))-1))
```

```
}
```

```
#PDF
```

```
curve(f(x,0.5,0.3,0.3,0.7),main="", ylab="pdf", xlab="X" ,ylim=c(0,3.1),0,1.8,lwd=2)
```

```
curve(f(x,0.3,0.9,0.5,0.9),lty=1,col=2,add=T,lwd=2)
```

```
curve(f(x,0.9,0.7,0.8,0.8),lty=1,col=3,add=T,lwd=2)
```

```
curve(f(x,0.7,0.6,0.5,1.1),lty=1,col=4,add=T,lwd=2)
```

```
curve(f(x,0.8,0.5,0.6,1.0),lty=1,col=5,add=T,lwd=2)
```

```
curve(f(x,0.6,0.7,0.5,0.7),lty=1,col=6,add=T,lwd=2)
```

```
legend("topright",title=expression(""),
```

```
  c(expression(lambda*"=0.50","*~omega*"=0.30 , "*~delta*"=0.30 , "*~zeta*"=0.70      ")),
```

```

expression(lambda*"=0.30,"*~omega*"=0.90,"*~delta*"=0.50,"*~zeta*"=0.90"),
expression(lambda*"=0.90,"*~omega*"=0.70,"*~delta*"=0.80,"*~zeta*"=0.80"),
expression(lambda*"=0.70,"*~omega*"=0.60,"*~delta*"=0.50,"*~zeta*"=1.10"),
expression(lambda*"=0.80,"*~omega*"=0.50,"*~delta*"=0.60,"*~zeta*"=1.00"),
expression(lambda*"=0.60,"*~omega*"=0.70,"*~delta*"=0.40,"*~zeta*"=0.70))),
cex=0.9,lty=c(1:1),lwd=2,col=c(1,2,3,4,5,6))

#CDF
curve(F(x,0.5,0.3,0.3,0.7),main="", ylab="CDF", xlab="X",ylim=c(0,1.05),0,2.0,lwd=2)
curve(F(x,0.3,0.9,0.5,0.9),lty=1,col=2,add=T,lwd=2)
curve(F(x,0.9,0.7,0.8,0.8),lty=1,col=3,add=T,lwd=2)
curve(F(x,0.7,0.6,0.5,1.1),lty=1,col=4,add=T,lwd=2)
curve(F(x,0.8,0.5,0.6,1.0),lty=1,col=5,add=T,lwd=2)
curve(F(x,0.6,0.7,0.5,0.7),lty=1,col=6,add=T,lwd=2)

legend("bottomright",title=expression(""),
      c(expression(lambda*"=0.50,"*~omega*"=0.30,"*~delta*"=0.30,"*~zeta*"=0.70
      ),
      expression(lambda*"=0.30,"*~omega*"=0.90,"*~delta*"=0.50,"*~zeta*"=0.90"),
      expression(lambda*"=0.90,"*~omega*"=0.70,"*~delta*"=0.80,"*~zeta*"=0.80"),
      expression(lambda*"=0.70,"*~omega*"=0.60,"*~delta*"=0.50,"*~zeta*"=1.10"),
      expression(lambda*"=0.80,"*~omega*"=0.50,"*~delta*"=0.60,"*~zeta*"=1.00"),
      expression(lambda*"=0.60,"*~omega*"=0.70,"*~delta*"=0.40,"*~zeta*"=0.70))),
cex=0.9,lty=c(1:1),lwd=2,col=c(1,2,3,4,5,6))

#SURVIVAL
curve(S(x,0.5,0.3,0.3,0.7),main="", ylab="Survival", xlab="X",ylim=c(0,1.005),0,2.0,lwd=2)
curve(S(x,0.3,0.9,0.5,0.9),lty=1,col=2,add=T,lwd=2)
curve(S(x,0.9,0.7,0.8,0.8),lty=1,col=3,add=T,lwd=2)

```

```
curve(S(x,0.7,0.6,0.5,1.1),lty=1,col=4,add=T,lwd=2)
```

```
curve(S(x,0.8,0.5,0.6,1.0),lty=1,col=5,add=T,lwd=2)
```

```
curve(S(x,0.6,0.7,0.5,0.7),lty=1,col=6,add=T,lwd=2)
```

```
legend("topright",title=expression(""),
```

```
  c(expression(lambda*"=0.50","*~omega*"=0.30 ,"*~delta*"=0.30 ,"*~zeta*"=0.70      ),
```

```
    expression(lambda*"=0.30","*~omega*"=0.90 ,"*~delta*"=0.50,"*~zeta*"=0.90"),
```

```
    expression(lambda*"=0.90","*~omega*"=0.70 ,"*~delta*"=0.80,"*~zeta*"=0.80"),
```

```
    expression(lambda*"=0.70","*~omega*"=0.60 ,"*~delta*"=0.50,"*~zeta*"=1.10"),
```

```
    expression(lambda*"=0.80","*~omega*"=0.50 ,"*~delta*"=0.60,"*~zeta*"=1.00"),
```

```
    expression(lambda*"=0.60","*~omega*"=0.70 ,"*~delta*"=0.40,"*~zeta*"=0.70))),
```

```
  cex=0.9,lty=c(1:1),lwd=2,col=c(1,2,3,4,5,6))
```

```
#Haz
```

```
curve(h(x,5.5,0.3,1.90,1.2),main="", ylab="Hazard", xlab="X" ,ylim=c(0,3.5),0,8.9,lwd=2)
```

```
curve(h(x,4.80,0.85,0.6,0.65),lty=1,col=2,add=T,lwd=2)
```

```
curve(h(x,3.90,0.50,0.9,0.3),lty=1,col=3,add=T,lwd=2)
```

```
curve(h(x,0.95,1.0,1.8,4.50),lty=1,col=4,add=T,lwd=2)
```

```
curve(h(x,2.0,0.5,0.6,1.90),lty=1,col=5,add=T,lwd=2)
```

```
curve(h(x,3.10,0.80,1.4,2.39),lty=1,col=6,add=T,lwd=2)
```

```
legend("topright",title=expression(""),
```

```
  c(expression(lambda*"=0.50","*~omega*"=0.30 ,"*~delta*"=0.30 ,"*~zeta*"=0.70      ),
```

```
    expression(lambda*"=0.30","*~omega*"=0.90 ,"*~delta*"=0.50,"*~zeta*"=0.90"),
```

```
    expression(lambda*"=0.90","*~omega*"=0.70 ,"*~delta*"=0.80,"*~zeta*"=0.80"),
```

```
    expression(lambda*"=0.70","*~omega*"=0.60 ,"*~delta*"=0.50,"*~zeta*"=1.10"),
```

```
    expression(lambda*"=0.80","*~omega*"=0.50 ,"*~delta*"=0.60,"*~zeta*"=1.00"),
```

```
    expression(lambda*"=0.60","*~omega*"=0.70 ,"*~delta*"=0.40,"*~zeta*"=0.70))),
```

```
cex=0.8,lty=c(1:1),lwd=2,col=c(1,2,3,4,5,6))
```

Application on real data

```
rm(list=ls())
```

```
library(Newdistns)
```

```
library(AdequacyModel)
```

```
x <- c( 3.70, 2.74, 2.73, 2.50, 3.60, 3.11, 3.27, 2.87, 1.47, 3.11, 4.42, 2.41, 3.19, 3.22, 1.69,  
3.28, 3.09,
```

```
1.87, 3.15, 4.90, 3.75, 2.43, 2.95, 2.97, 3.39, 2.96, 2.53, 2.67, 2.93, 3.22, 3.39, 2.81,  
4.20, 3.33, 2.55, 3.31, 3.31,
```

```
2.85, 2.56, 3.56, 3.15, 2.35, 2.55, 2.59, 2.38, 2.81, 2.77, 2.17, 2.83, 1.92, 1.41, 3.68,  
2.97, 1.36, 0.98, 2.76, 4.91,
```

```
3.68, 1.84, 1.59, 3.19, 1.57, 0.81, 5.56, 1.73, 1.59, 2.00, 1.22, 1.12, 1.71, 2.17, 1.17,  
5.08, 2.48, 1.18, 3.51, 2.17,
```

```
1.69, 1.25, 4.38, 1.84, 0.39, 3.68, 2.48, 0.85, 1.61, 2.79, 4.70, 2.03, 1.80, 1.57, 1.08,  
2.03, 1.61, 2.12, 1.89, 2.88,
```

```
2.82, 2.05, 3.65 )
```

```
Go <- function (g, data, starts, method = "BFGS")
```

```
{
```

```
  if ( g != "e" )
```

```
  {
```

```
    stop("Baseline distribution not implemented or misspelled. Please check the manual for  
guidelines.")
```

```
  }
```

```

if (g == "e") {
  den = function(par, x) {
    b = par[3]
    c = par[4]
    2*b^2*c*x^(-3)*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1)
  }
  cum = function(par, x) {
    b = par[3]
    c = par[4]
    1-(1-exp(-b^2*x^(-2)))^c
  }
}

```

```

pdf0 <- function(par, x) {
  f0 = den(par, x)
  c0 = cum(par, x)
  r = par[1]
  u = par[2]
  r*(f0/(1-c0)^2)*exp(u*(c0/(1-c0)))*exp(-(r/u)*(exp(u*c0/(1-c0))-1))
}

```

```

cdf0 <- function(par, x) {
  f0 = den(par, x)
  c0 = cum(par, x)
  r = par[1]
  u = par[2]

```

```

1-exp(-(r/u)*(exp(u*c0/(1-c0))-1))

}

med = suppressWarnings(goodness.fit(pdf = pdf0, cdf = cdf0,
                                   starts = starts, data = data, method = method, mle = NULL))

aux = cbind(med$Erro, med$mle + qnorm(0.025) * med$Erro,
            med$mle + qnorm(0.975) * med$Erro)

colnames(aux) = c("Std. Dev.", "Inf. 95% CI", "Sup. 95% CI")

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,
             med$A)

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,
             med$A)

colnames(aux1) = c("AIC", "CAIC", "BIC", "HQIC", "W", "A")

rownames(aux1) = c("")

aux5 = cbind(med$mle)

colnames(aux5) = c("MLE")

aux4 = cbind(med$Value)

colnames(aux4) = c("Min(-log(Likelihood))")

rownames(aux4) = c("")

aux2 = cbind(med$KS$statistic, med$KS$p.value)

colnames(aux2) = c("KS Statistic", "KS p-value")

rownames(aux2) = c("")

aux3 = cbind(if (med$Convergence == 0) {
  "Algorithm Converged"
}

else {

```

```

    "Algorithm Not Converged"
  })
  colnames(aux3) = c("")
  rownames(aux3) = c("")
  list(Estimates = aux, Measures = aux1, `Kolmogorov-Smirnov Test` = aux2,
    `Convergence Status` = aux3, `Loglike`=aux4, `Parameter`=aux5)
}

```

```
#####
```

```
##### Nedragha #####
```

```

OL <- function (g, data, starts, method = "BFGS")
{
  if ( g != "e" )
  {
    stop("Baseline distribution not implemented or misspelled. Please check the manual for
guidelines.")
  }
  if (g == "e") {
    den = function(par, x) {
      b = par[3]
      c = par[4]
       $2 * b^2 * c * x^{-3} * \exp(-b^2 * x^{-2}) * (1 - \exp(-b^2 * x^{-2}))^{(c-1)}$ 
    }
    cum = function(par, x) {
      b = par[3]
      c = par[4]
       $1 - (1 - \exp(-b^2 * x^{-2}))^c$ 
    }
  }
}

```

```

}
}

```

```

pdf0 <- function(par, x) {
  f0 = den(par, x)
  c0 = cum(par, x)
  r = par[1]
  u = par[2]
  (u/r)*f0*((1-(c0*log(1 - c0)*r^(-1)))^(-u-1))*((c0/(1-c0))-log(1 - c0))
}

```

```

cdf0 <- function(par, x) {
  f0 = den(par, x)
  c0 = cum(par, x)
  r = par[1]
  u = par[2]
  1 - (1-(c0*log(1 - c0))/r)^(-u)
}

```

```

med = suppressWarnings(goodness.fit(pdf = pdf0, cdf = cdf0,
                                   starts = starts, data = data, method = method, mle = NULL))
aux = cbind(med$Erro, med$mle + qnorm(0.025) * med$Erro,
            med$mle + qnorm(0.975) * med$Erro)
colnames(aux) = c("Std. Dev.", "Inf. 95% CI", "Sup. 95% CI")
aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,
            med$A)
aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,
            med$A)
colnames(aux1) = c("AIC", "CAIC", "BIC", "HQIC", "W", "A")

```



```

rownames(aux1) = c("")
aux5 = cbind(med$mle)
colnames(aux5) = c("MLE")
aux4 = cbind(med$Value)
colnames(aux4) = c( "Min(-log(Likelihood))")
rownames(aux4) = c("")
aux2 = cbind(med$KS$statistic, med$KS$p.value)
colnames(aux2) = c("KS Statistic", "KS p-value")
rownames(aux2) = c("")
aux3 = cbind(if (med$Convergence == 0) {
  "Algorithm Converged"
}
else {
  "Algorithm Not Converged"
})
colnames(aux3) = c("")
rownames(aux3) = c("")
list(Estimates = aux, Measures = aux1, `Kolmogorov-Smirnov Test` = aux2,
     `Convergence Status` = aux3, `Loglike`=aux4, `Parameter`=aux5)
}

#####Ku exp
ku <- function (g, data, starts, method = "BFGS")
{
  if ( g != "e" )
  {
    stop("Baseline distribution not implemented or misspelled. Please check the manual for
guidelines.")
  }
}

```

```

}

if (g == "e") {

  den = function(par, x) {

    b = par[3]

    c = par[4]

     $2 * b^2 * c * x^{(-3)} * \exp(-b^2 * x^{(-2)}) * (1 - \exp(-b^2 * x^{(-2)}))^{(c-1)}$ 

  }

  cum = function(par, x) {

    b = par[3]

    c = par[4]

     $1 - (1 - \exp(-b^2 * x^{(-2)}))^{(c)}$ 

  }

}

```

```

pdf0 <- function(par, x) {

  f0 = den(par, x)

  c0 = cum(par, x)

  r = par[1]

  u = par[2]

   $r * u * f0 * (c0^{(r - 1)}) * (1 - c0^r)^{(u - 1)}$ 

}

```

```

cdf0 <- function(par, x) {

  f0 = den(par, x)

  c0 = cum(par, x)

  r = par[1]

  u = par[2]

   $1 - (1 - c0^r)^u$ 

}

```

```

}

med = suppressWarnings(goodness.fit(pdf = pdf0, cdf = cdf0,

                                starts = starts, data = data, method = method, mle = NULL))

aux = cbind(med$Erro, med$mle + qnorm(0.025) * med$Erro,

            med$mle + qnorm(0.975) * med$Erro)

colnames(aux) = c("Std. Dev.", "Inf. 95% CI", "Sup. 95% CI")

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,

            med$A)

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,

            med$A)

colnames(aux1) = c("AIC", "CAIC", "BIC", "HQIC", "W", "A")

rownames(aux1) = c("")

aux5 = cbind(med$mle)

colnames(aux5) = c("MLE")

aux4 = cbind(med$Value)

colnames(aux4) = c("Min(-log(Likelihood))")

rownames(aux4) = c("")

aux2 = cbind(med$KS$statistic, med$KS$p.value)

colnames(aux2) = c("KS Statistic", "KS p-value")

rownames(aux2) = c("")

aux3 = cbind(if (med$Convergence == 0) {

    "Algorithm Converged"

}

else {

    "Algorithm Not Converged"

})

colnames(aux3) = c(" ")

```

```

rownames(aux3) = c("")

list(Estimates = aux, Measures = aux1, `Kolmogorov-Smirnov Test` = aux2,
     `Convergence Status` = aux3, `Loglike`=aux4, `Parameter`=aux5)
}

##### EG exp

eg <- function (g, data, starts, method = "BFGS")
{
  if ( g != "e" )
  {
    stop("Baseline distribution not implemented or misspelled. Please check the manual for
guidelines.")
  }
  if (g == "e") {
    den = function(par, x) {
      b = par[3]
      c = par[4]
      
$$2*b^2*c*x^{(-3)}*exp(-b^2*x^{(-2)})*(1-exp(-b^2*x^{(-2)}))^{(c-1)}$$

    }
    cum = function(par, x) {
      b = par[3]
      c = par[4]
      
$$1-(1-exp(-b^2*x^{(-2)}))^{(c)}$$

    }
  }

  pdf0 <- function(par, x) {

```

```

f0 = den(par, x)

c0 = cum(par, x)

r = par[1]

u = par[2]

r * u * f0 * ((1 - c0)^(r - 1)) * (1 - (1 - c0)^r)^(u -
1)

}

cdf0 <- function(par, x) {

  f0 = den(par, x)

  c0 = cum(par, x)

  r = par[1]

  u = par[2]

  (1 - (1 - c0)^r)^u

}

med = suppressWarnings(goodness.fit(pdf = pdf0, cdf = cdf0,

starts = starts, data = data, method = method, mle = NULL))

aux = cbind(med$Erro, med$mle + qnorm(0.025) * med$Erro,

med$mle + qnorm(0.975) * med$Erro)

colnames(aux) = c("Std. Dev.", "Inf. 95% CI", "Sup. 95% CI")

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,

med$A)

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,

med$A)

colnames(aux1) = c("AIC", "CAIC", "BIC", "HQIC", "W", "A")

rownames(aux1) = c("")

aux5 = cbind(med$mle)

colnames(aux5) = c("MLE")

```

```

aux4 = cbind(med$Value)

colnames(aux4) = c( "Min(-log(Likelihood))")

rownames(aux4) = c("")

aux2 = cbind(med$KS$statistic, med$KS$p.value)

colnames(aux2) = c("KS Statistic", "KS p-value")

rownames(aux2) = c("")

aux3 = cbind(if (med$Convergence == 0) {

  "Algorithm Converged"

}

else {

  "Algorithm Not Converged"

}))

colnames(aux3) = c("")

rownames(aux3) = c("")

list(Estimates = aux, Measures = aux1, `Kolmogorov-Smirnov Test` = aux2,

  `Convergence Status` = aux3, `Loglike`=aux4, `Parameter`=aux5)

}

```

MO

```

LGam <- function (g, data, starts, method = "BFGS")

{

  if ( g != "e" )

  {

    stop("Baseline distribution not implemented or misspelled. Please check the manual for
guidelines.")

  }

  if (g == "e") {

    den = function(par, x) {

```

```

b = par[3]

c = par[4]

2*b^2*c*x^(-3)*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1)
}

cum = function(par, x) {

  b = par[3]

  c = par[4]

  1-(1-exp(-b^2*x^(-2)))^c

}
}

```

```

pdf0 <- function(par, x) {

  f0 = den(par, x)

  c0 = cum(par, x)

  r = par[1]

  u = par[2]

  u^r * gamma(r)^(-1) * f0 * (-log(1 - c0))^(r - 1) * (1 -
                                c0)^(u - 1)

}

```

```

cdf0 <- function(par, x) {

  f0 = den(par, x)

  c0 = cum(par, x)

  r = par[1]

  u = par[2]

  1 - pgamma(-u * log(1 - c0), shape = r)

}

```

```

med = suppressWarnings(goodness.fit(pdf = pdf0, cdf = cdf0,

```

```

starts = starts, data = data, method = method, mle = NULL))

aux = cbind(med$Erro, med$mle + qnorm(0.025) * med$Erro,
            med$mle + qnorm(0.975) * med$Erro)

colnames(aux) = c("Std. Dev.", "Inf. 95% CI", "Sup. 95% CI")

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,
            med$A)

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,
            med$A)

colnames(aux1) = c("AIC", "CAIC", "BIC", "HQIC", "W", "A")

rownames(aux1) = c("")

aux5 = cbind(med$mle)

colnames(aux5) = c("MLE")

aux4 = cbind(med$Value)

colnames(aux4) = c("Min(-log(Likelihood))")

rownames(aux4) = c("")

aux2 = cbind(med$KS$statistic, med$KS$p.value)

colnames(aux2) = c("KS Statistic", "KS p-value")

rownames(aux2) = c("")

aux3 = cbind(if (med$Convergence == 0) {
  "Algorithm Converged"
}
else {
  "Algorithm Not Converged"
})

colnames(aux3) = c("")

rownames(aux3) = c("")

list(Estimates = aux, Measures = aux1, `Kolmogorov-Smirnov Test` = aux2,

```



```

`Convergence Status` = aux3, `Loglike`=aux4, `Parameter`=aux5)
}

##### Beta - exp

Be <- function (g, data, starts, method = "BFGS") ##### new -G
{
  if ( g != "e" )
  {
    stop("Baseline distribution not implemented or misspelled. Please check the manual for
guidelines.")
  }

  if (g == "e") {
    den = function(par, x) {
      b = par[3]
      c = par[4]
      
$$2*b^2*c*x^{(-3)}*exp(-b^2*x^{(-2)})*(1-exp(-b^2*x^{(-2)}))^{(c-1)}$$

    }

    cum = function(par, x) {
      b = par[3]
      c = par[4]
      
$$1-(1-exp(-b^2*x^{(-2)}))^c$$

    }
  }

  pdf0 <- function(par, x) {
    f0 = den(par, x)
    c0 = cum(par, x)
  }
}

```

```

r = par[1]

u = par[2]

f0 * beta(r, u)^(-1) * c0^(r - 1) * (1 - c0)^(u - 1)
}

cdf0 <- function(par, x) {

  f0 = den(par, x)

  c0 = cum(par, x)

  r = par[1]

  u = par[2]

  pbeta(c0, shape1 = r, shape2 = u)

}

med = suppressWarnings(goodness.fit(pdf = pdf0, cdf = cdf0,

                                starts = starts, data = data, method = method, mle = NULL))

aux = cbind(med$Erro, med$mle + qnorm(0.025) * med$Erro,

            med$mle + qnorm(0.975) * med$Erro)

colnames(aux) = c("Std. Dev.", "Inf. 95% CI", "Sup. 95% CI")

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,

            med$A)

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,

            med$A)

colnames(aux1) = c("AIC", "CAIC", "BIC", "HQIC", "W", "A")

rownames(aux1) = c("")

aux5 = cbind(med$mle)

colnames(aux5) = c("MLE")

aux4 = cbind(med$Value)

```

```

colnames(aux4) = c( "Min(-log(Likelihood))")
rownames(aux4) = c("")
aux2 = cbind(med$KS$statistic, med$KS$p.value)
colnames(aux2) = c("KS Statistic", "KS p-value")
rownames(aux2) = c("")
aux3 = cbind(if (med$Convergence == 0) {
  "Algorithm Converged"
}
else {
  "Algorithm Not Converged"
})
colnames(aux3) = c("")
rownames(aux3) = c("")
list(Estimates = aux, Measures = aux1, `Kolmogorov-Smirnov Test` = aux2,
     `Convergence Status` = aux3, `Loglike`=aux4, `Parameter`=aux5)
}

```

```
##### Gom
```

```
#density of Gom
```

```
pdf_Exp <- function(par,x) {
  b = par[1]
  c = par[2]
  2*b^2*c*x^(-3)*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1)
}

```

```
#cdf of Gom
```

```
cdf_Exp <- function(par,x) {

```

```

b = par[1]

c = par[2]

1-(1-exp(-b^2*x^(-2)))^c

}

med = suppressWarnings(goodness.fit(pdf=pdf_Exp, cdf=cdf_Exp, starts=c(0.01,0.1), data=x,
method = "B",

                        domain = c(0,Inf),mle = NULL))

aux = cbind(med$Erro, med$mle + qnorm(0.025) * med$Erro,

            med$mle + qnorm(0.975) * med$Erro)

colnames(aux) = c("Std. Dev.", "Inf. 95% CI", "Sup. 95% CI")

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,

            med$A)

aux1 = cbind(med$AIC, med$CAIC, med$BIC, med$HQIC, med$W,

            med$A)

colnames(aux1) = c("AIC", "CAIC", "BIC", "HQIC", "W", "A")

rownames(aux1) = c("")

aux5 = cbind(med$mle)

colnames(aux5) = c("MLE")

aux4 = cbind(med$Value)

colnames(aux4) = c("Min(-log(Likelihood))")

rownames(aux4) = c("")

aux2 = cbind(med$KS$statistic, med$KS$p.value)

colnames(aux2) = c("KS Statistic", "KS p-value")

rownames(aux2) = c("")

aux3 = cbind(if (med$Convergence == 0) {

"Algorithm Converged"

```

```

}

else {

  "Algorithm Not Converged"

})

colnames(aux3) = c("")

rownames(aux3) = c("")

MUN7=list(Estimates = aux, Measures = aux1, `Kolmogorov-Smirnov Test` = aux2,
          `Convergence Status` = aux3, `Loglike`=aux4, `Parameter`=aux5)

MUN7

#####

library(psych)

describe(x)


MUN1=Go("e",x,starts=c(1,0.1,0.1,0.1),method="B")

MUN1

MUN2=OL("e",x,starts=c(1,0.1,0.1,0.1),method="B")

MUN2

MUN3=ku("e",x,starts=c(1,0.1,0.1,0.1),method="B")

MUN3

MUN4=eg("e",x,starts=c(1,0.1,0.1,0.1),method="B")

MUN4

MUN5=LGam("e",x,starts=c(1,0.1,0.1,0.1),method="B")

MUN5

MUN6=Be("e",x,starts=c(1,0.1,0.1,0.1),method="B")

MUN6

MUN7

```

#####

MUN1

MUN2

MUN3

MUN4

MUN5

MUN6

MUN7

#####

parGoE=MUN1\$Parameter

parGoE

parOLE=MUN2\$Parameter

parOLE

parKuE=MUN3\$Parameter

parKuE

parEGE=MUN4\$Parameter

parEGE

parLGamE=MUN5\$Parameter

parLGamE

parBeE=MUN6\$Parameter

parBeE

parE=MUN7\$Parameter

parE

#####33

StaGoE=MUN1\$Measures

StaOLE=MUN2\$Measures

StaKuE=MUN3\$Measures

StaEGE=MUN4\$Measures

StaLGame=MUN5\$Measures

StaBeE=MUN6\$Measures

StaE=MUN7\$Measures

#####

StaGoE

StaOLE

StaKuE

StaEGE

StaLGame

StaBeE

StaE

#####

```
fGoE <- function(x,r,u,b,c){
```

```
  ff=r*((2*b^2*c*x^(-3)*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1))/(1-(1-(1-exp(-b^2*x^(-2)))^c))^2)*exp(u*((1-(1-exp(-b^2*x^(-2)))^c)/(1-(1-(1-exp(-b^2*x^(-2)))^c))))*exp(-(r/u)*(exp(u*(1-(1-exp(-b^2*x^(-2)))^c)/(1-(1-(1-exp(-b^2*x^(-2)))^c)))-1))
```

```
}
```

```
fOLE <- function(x,r,u,b,c){
```

```
  ff=(u/r)*(2*b^2*c*x^(-3)*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1))*(((1-(1-(1-exp(-b^2*x^(-2)))^c)*log(1 - (1-(1-exp(-b^2*x^(-2)))^c))*r^(-1)))^(-u-1))*(((1-(1-exp(-b^2*x^(-2)))^c)/(1-(1-(1-exp(-b^2*x^(-2)))^c)))-log(1 - (1-(1-exp(-b^2*x^(-2)))^c)))
```

```
}
```

```
fKuE <- function(x,r,u,b,c){
```

```
  ff=r * u * (2*b^2*c*x^(-3)*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1)) * (((1-(1-exp(-b^2*x^(-2)))^c))^(r - 1)) * (1 - (1-(1-exp(-b^2*x^(-2)))^c)^r)^(u - 1)
```

```
}
```

```

fEGE <- function(x,r,u,b,c){

  ff=r * u * (2*b^2*c*x^(-3)*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1)) * ((1 - (1-(1-exp(-
b^2*x^(-2))))^c))^(r - 1)) * (1 - (1 - (1-(1-exp(-b^2*x^(-2))))^c))^r^(u - 1)}

fLGamE <- function(x,r,u,b,c){

  ff = u^r * gamma(r)^(-1) * (2*b^2*c*x^(-3)*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1)) *
(-log(1 - (1-(1-exp(-b^2*x^(-2))))^c))^(r - 1) * (1 - (1-(1-exp(-b^2*x^(-2))))^c)^(u - 1)}

fBeE <- function(x,r,u,b,c){

  ff = (2*b^2*c*x^(-3)*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1)) * beta(r, u)^(-1) * (1-(1-
exp(-b^2*x^(-2))))^c)^(r - 1) * (1 - (1-(1-exp(-b^2*x^(-2))))^c)^(u - 1)

}

fE <- function(x,b,c){

  ff =2*b^2*c*x^(-3)*exp(-b^2*x^(-2))*(1-exp(-b^2*x^(-2)))^(c-1)

}

#####

x1<-seq(0,max(x+1),0.01)

hist(x,freq=F,, col = "pink",xlim=c(0,7),ylim=c(0,0.6),main=" ",xlab="x",ylab=" f(x)")

box()

lines(x1,fGoE(x1,parGoE[1],parGoE[2],parGoE[3],parGoE[4]),lty=1,lwd=2,col=1)

lines(x1,fOLE(x1,parOLE[1],parOLE[2],parOLE[3],parOLE[4]),lty=1,lwd=2,col=2)

lines(x1,fKuE(x1,parKuE[1],parKuE[2],parKuE[3],parKuE[4]),lty=1,lwd=2,col=3)

lines(x1,fEGE(x1,parEGE[1],parEGE[2],parEGE[3],parEGE[4]),lty=1,lwd=2,col=4)

lines(x1,fLGamE(x1,parLGamE[1],parLGamE[2],parLGamE[3],parLGamE[4]),lty=1,lwd=2,col=5
)

lines(x1,fBeE(x1,parBeE[1],parBeE[2],parBeE[3],parBeE[4]),lty=1,lwd=2,col=6)

lines(x1,fE(x1,parE[1],parE[2]),lty=1,lwd=2,col=8)


legend("topright" ,cex=0.9 ,
legend=c("EGIBX","OLIBX","KuIBX","EEGIBX","LGamIBX","BeIBX","IBX"),lty=c(1,1,1,1,1,1,1,
2),lwd=2, col=c(1,2,3,4,5,6,7,8,9))

#####3

```



```

FGoE <- function(x,r,u,b,c){

ff = 1-exp(-(r/u)*(exp(u*(1-(1-exp(-b^2*x^(-2)))^c)/(1-(1-(1-exp(-b^2*x^(-2)))^c)))-1))

}

```

```

FOLE <- function(x,r,u,b,c){

ff = 1 - (1-((1-(1-exp(-b^2*x^(-2)))^c)*log(1 - (1-(1-exp(-b^2*x^(-2)))^c)))/r)^(-u)}

```

```

FKuE <- function(x,r,u,b,c){

ff = 1 - (1 - (1-(1-exp(-b^2*x^(-2)))^c)^r)^u }

```

```

FEGE <- function(x,r,u,b,c){

ff =(1 - (1 - (1-(1-exp(-b^2*x^(-2)))^c))^r)^u }

```

```

FLGamE <- function(x,r,u,b,c){

ff = pgamma(-u * log(1 - (1-(1-exp(-b^2*x^(-2)))^c)), shape = r)}

```

```

FBeE <- function(x,r,u,b,c){

ff=pbeta(1-(1-exp(-b^2*x^(-2)))^c, shape1 = r, shape2 = u) }

```

```

FE <- function(x,b,c){

ff =1-(1-exp(-b^2*x^(-2)))^c

}

```

```
#####
```

```

x1 <- seq(0,100,length.out=1000)

plot(ecdf(x),lty=1, lwd=1, do.points=FALSE, verticals=TRUE, main="")

lines(x1,FGoE(x1,parGoE[1],parGoE[2],parGoE[3],parGoE[4]),lty=1,lwd=2,col=1)

lines(x1,FOLE(x1,parOLE[1],parOLE[2],parOLE[3],parOLE[4]),lty=1,lwd=2,col=2)

lines(x1,FKuE(x1,parKuE[1],parKuE[2],parKuE[3],parKuE[4]),lty=1,lwd=2,col=3)

lines(x1,FEGE(x1,parEGE[1],parEGE[2],parEGE[3],parEGE[4]),lty=1,lwd=2,col=4)

```

```
lines(x1,FLGamE(x1,parLGamE[1],parLGamE[2],parLGamE[3],parLGamE[4]),lty=1,lwd=2,col=
5)
```

```
lines(x1,FBeE(x1,parBeE[1],parBeE[2],parBeE[3],parBeE[4]),lty=1,lwd=2,col=6)
```

```
lines(x1,FE(x1,parE[1],parE[2]),lty=1,lwd=2,col=8)
```

```
legend("bottomright",cex=0.9 ,
legend=c("EGIBX","OLIBX","KuIBX","EEGIBX","LGamIBX","BeIBX","IBX"),lty=c(1,1,1,1,1,1,1,
2),lwd=1, col=c(1,2,3,4,5,6,7,8,9))
```

```
#####3
```

Simulation

```
remove(list=objects())
```

```
options(warn = -1)
```

```
fx <- function(x,theta,beta,eta,kappa){theta*((2*eta^2*kappa*x^(-3)*exp(-eta^2*x^(-2))*(1-
exp(-eta^2*x^(-2)))^(kappa-1))/(1-(1-(1-exp(-eta^2*x^(-2)))^kappa))^2)*exp(beta*((1-(1-
exp(-eta^2*x^(-2)))^kappa)/(1-(1-(1-exp(-eta^2*x^(-2)))^kappa))))*exp(-
(theta/beta)*(exp(beta*(1-(1-exp(-eta^2*x^(-2)))^kappa)/(1-(1-(1-exp(-eta^2*x^(-
2)))^kappa))-1)))}
```

```
Fx <- function(x,theta,beta,eta,kappa){1-exp(-(theta/beta)*(exp(beta*(1-(1-exp(-eta^2*x^(-
2)))^kappa)/(1-(1-(1-exp(-eta^2*x^(-2)))^kappa))-1)))}
```

```
Sx <- function(x,theta,beta,eta,kappa){1-Fx(x,theta,beta,eta,kappa)}
```

```

xp <- function(u,theta,beta,eta,kappa){

F1 = function(x){Fx(x,theta,beta,eta,kappa)}

Inv = function(u){uniroot(function(x){F1(x) - u}, c(0,10000),
                           extendInt = "yes")$root}

## generating data

h = c()

for(j in 1:length(u)){

  h[j] = Inv(u[j])

}

return(h)

}

## Negative log-likelihood function

Nlog_like = function(parm){

  theta  = parm[1]

  beta   = parm[2]

  eta    = parm[3]

  kappa  = parm[4]

  Nlog_like = -sum(log(fx(x,theta,beta,eta,kappa)))

  return(Nlog_like)

}

```

```
## least square
```

```
LS = function(parm){  
  theta = parm[1]  
  beta = parm[2]  
  eta = parm[3]  
  kappa = parm[4]  
  x = sort(x)  
  LS = sum((Fx(x,theta,beta,eta,kappa)-((1:n)/(n+1)))^2)  
  return(LS)  
}
```

```
## Weighted least square method
```

```
WLS = function(parm){  
  theta = parm[1]  
  beta = parm[2]  
  eta = parm[3]  
  kappa = parm[4]  
  j=1:n  
  x = sort(x)  
  w=(n+1)^2*(n+2)/(j*(n-j+1))  
  WLS = sum(w*(Fx(x,theta,beta,eta,kappa)-((j)/(n+1)))^2)  
  return(WLS)  
}
```

```
## function of resluts
```

```
Rslu = function(Estimate,initial){
```

```

Mean = mean(Estimate, na.rm = TRUE)

MSE = mean((Estimate-initial)^2, na.rm = TRUE)

RMSE = sqrt(MSE)

Bais = abs(Mean-initial)

Rslu = c(initial, Mean, MSE, RMSE,Bais)

}

```

```

####-----

```

```

## Parmaeters

```

```

## sample size

```

```

N = c(25, 50 , 100 , 200)

```

```

theta = 1.7

```

```

beta = 1.5

```

```

eta = 1.6

```

```

kappa = 1.8

```

```

####-----

```

```

## simuation

```

```

for(k in 1:4){

```

```

  ## sample size

```

```

  n = N[k]

```

```

  ## saving vectors

```

```

  N.sim = 2000

```

```

  MLE = matrix(NA, nrow = N.sim, ncol = 4)

```

```

  LSE = matrix(NA, nrow = N.sim, ncol = 4)

```

```

  WLSE = matrix(NA, nrow = N.sim, ncol = 4)

```

```

  for(i in 1:N.sim){

```

```

## for fixed random generating

# set.seed(i)


## generating uinform

u = runif(n)

x = xp(u,theta,beta,eta,kappa)


## Maximu likelihood estimation

fit_mle = optim(c(theta=theta,beta=beta,eta=eta,kappa=kappa), fn = Nlog_like, hessian =
TRUE, method = "N")

if(fit_mle$convergence==1){MLE[i,] = c(NA,NA)}

else{ MLE[i,] = fit_mle$par}


## Least square estimation

fit_ls = optim(c(theta=theta,beta=beta,eta=eta,kappa=kappa), fn = LS, hessian = TRUE,
method = "N")

# LSE[i,] = fit_ls

if(fit_ls$convergence==1){LSE[i,] = c(NA,NA)}

else{ LSE[i,] = fit_ls$par}


## Weighted Least square estimation

fit_wlse = optim(c(theta=theta,beta=beta,eta=eta,kappa=kappa), fn = WLS, hessian =
TRUE, method = "N")

# WLSE[i,] = fit_wlse

if(fit_wlse$convergence==1){WLSE[i,] = c(NA,NA)}

else{ WLSE[i,] = fit_wlse$par}

```

```
}
```

```
## estimate uinsg Monte-Carlo: MLE
```

```
theta_mle = Rslu(MLE[,1],theta)
```

```
beta_mle = Rslu(MLE[,2],beta)
```

```
eta_mle = Rslu(MLE[,3],eta)
```

```
kappa_mle = Rslu(MLE[,4],kappa)
```

```
## estimate uinsg Monte-Carlo: LSE
```

```
theta_ls = Rslu(LSE[,1],theta)
```

```
beta_ls = Rslu(LSE[,2],beta)
```

```
eta_ls = Rslu(LSE[,3],eta)
```

```
kappa_ls = Rslu(LSE[,4],kappa)
```

```
## estimate uinsg Monte-Carlo: WLSE
```

```
theta_wlse = Rslu(WLSE[,1],theta)
```

```
beta_wlse = Rslu(WLSE[,2],beta)
```

```
eta_wlse = Rslu(WLSE[,3],eta)
```

```
kappa_wlse = Rslu(WLSE[,4],kappa)
```

```
Name = c("initial","Mean", "MSE","RMSE" ,"Bias")
```

```
Reslutofsimulation = data.frame(Name, theta_mle, beta_mle, eta_mle, kappa_mle,  
theta_ls, beta_ls, eta_ls, kappa_ls, theta_wlse, beta_wlse, eta_wlse, kappa_wlse)
```

```
Summary = data.frame("Sample size"=n, "N.Simulation"=N.sim, "Initial theta"= theta,  
"Initial beta"= beta, "Initial eta"= eta,
```

```
"Initial kappa"= kappa)
```

```
## print reslut
```

```
Summary
```

```
Reslutofsimulation
```

```
print(k)
```

```
print(Summary)
```

```
print(Reslutofsimulation)
```

```
}
```